Задача 1. Выравнивание структур

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Петя изучает язык C, и только что дошёл до структур. Его особенно заинтересовал тот факт, что поля структуры не всегда размещаются в памяти подряд: иногда между соседними полями появляются дополнительные «пустоты» (padding). Из-за этого получается, что размер структуры может зависеть от того, в каком порядке программист перечислит поля структуры! Очень интересна в этом плане, например, вот эта статья.

Дано описание структуры на языке C. В первой строке написано ключевое слово struct и открывающая фигурная скобка. В последней строке записана закрывающая фигурная скобка и точка с запятой. Каждая строка между ними описывает ровно одно поле структуры. В описании поля сначала стоит тип поля, затем имя поля, и наконец точка с запятой. Тип поля отделён от имени поля хотя бы одним пробелом. Имя поля — это непустая строка, состоящая из латинских букв любого регистра, цифр и символов подчёркивания, причём имя точно не начинается с цифры.

В качестве типа поля может быть указан либо примитивный тип, либо указатель на примитивный тип. Во входных данных могут быть только следующие примитивные типы: char, short, int, long, float, int64_t и double. Указатель может быть любого порядка (т.е. двойной указатель, тройной указатель и т.п.). В данной задаче будем считать, что структура размещается в памяти следующим образом (в реальности правила размещения определяются компилятором). Первое поле структуры помещается по адресу с максимальным выравниванием: будем считать, что его адрес делится нацело на 8. Каждое следующее поле размещается по такому адресу, что:

1. оно расположено после предыдущего,
2. оно корректно выровнено, то есть его адрес делится на его размер, и
3. его адрес минимален при выполнении первых двух условий.

При этом между соседними полями может появиться пустое место (padding) размером от 1 до 7 байт.

После последнего поля также может быть добавлено несколько байт пустоты. Это необходимо, чтобы можно было хранить массив таких структур, то есть располагать в памяти много одинаковых структур друг за другом. В конце структуры добавляется минимальное количество байт пустоты, так чтобы в массиве любой длины все поля всех структур были корректно выровнены. При этом размер всей структуры равен сумме размеров всех полей и размеров всех имеющихся в структуре пустот.

Петя может переставлять местами поля структуры. Он хочет узнать, какой при этом может получиться минимальный размер структуры, и какой максимальный размер. Как известно, размер также зависит от модели данных. Петя хочет узнать ответ для всех популярных моделей.

Формат входного файла

Во входном файле описана одна структура согласно указанным в условии правилам. В любом месте описания может быть добавлено любое количество пробелов (ASCII 32), если их добавление не разрывает на части название примитивного типа, имя поля или ключевое слово.

Формат выходного файла

В выходном файле должно быть четыре строки, по одной строке на модель данных.
В каждой строке нужно записать через пробел два целых числа: минимально возможный размер структуры и максимально возможный размер.
В первой строке должен быть ответ для модели данных LP32, во второй — для модели ILP32, в третьей — для модели LLP64 и в четвёртой — для модели LP64. Информацию о том, сколько байтов занимает каждый тип на каждой модели данных, можно найти на странице.

Пример

input.txt	output.txt

struct
{
  int x;
  int64_t **Y;
  char _temp1;
};
	8 12
	12 12
	16 24
	16 24

Пояснение к примеру

Рассмотрим структуру в модели принятой на Win64 модели данных LLP64.
Поле x имеет размер 4 байта, поле Y размера 8 байт, а поле _temp1 занимает 1 байт. Если не менять порядок полей, тогда поле x будет занимать байты 0-3. Придётся добавить 4 байта пустоты, чтобы поле Y было выровнено по 8 байтам, заняв байты 8-15. Поле _temp1 будет иметь адрес 16 и занимать один байт, но после него нужно добавить ещё 7 байтов пустоты. Без этого поле Y второй структуры в массиве не может быть корректно выровнено. Получается общий размер 24 байта.
Если выбирать порядок полей, то размер 24 байта получается только когда поле Y находится между другими двумя полями. Так получается в 2 способах среди всех 6 спопобов выбора порядка. В остальных случаях размер структуры равен 16 байтам.

Задача 2. Односвязный список

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Требуется реализовать односвязный список на массиве. В каждом узле списка хранится строковое значение и индекс следующего узла. Дано начальное состояние списка, а также последовательность операций двух видов: добавление и удаление узла. Нужно выполнить все операции, и после этого вывести значения всех узлов списка в порядке их следования в цепи.

У каждого узла списка есть индекс в едином массиве, в котором всё хранится. В начальном состоянии списка есть N узлов, имеющих индексы от 0 до $N - 1$ в порядке их задания. При добавлении нового узла он дописывается в конец массива. То есть первый добавленный узел имеет индекс N, второй добавленный — $N + 1$, и так далее.

Нужно выполнять операции двух видов:

0. *Добавление узла.* При этом указывается индекс узла, после которого нужно вставить новый узел, и строковое значение для нового узла. Если индекс равен -1, то узел надо вставить в начало списка (перед самым первым элементом). После выполнения операции нужно вывести индекс нового узла (они назначаются по порядку, см. выше).

1. *Удаление узла.* При этом указывается индекс узла, и удалить нужно тот узел, который идёт сразу после него. Гарантируется, что узел с указанным индексом не последний. Если указан индекс -1, значит нужно удалить самый первый элемент списка (гарантируется, что он есть). После выполнения операции нужно вывести строковое значение удалённого узла.

В задаче используется “мультитест”: в одном входном файле записано много отдельных тестов.

Формат входного файла

В первой строке файла записано одно целое число T — количество тестов в файле. Далее в файле идут тесты (T штук) подряд, один за другим.
Первая строка теста начинается с трёх целых чисел: N — изначальное количество узлов в связном списке, F — индекс первого элемента списка и Q — количество операций, которые нужно выполнить ($0 \le F < N \le 10^5$, $0 \le Q \le 10^5$).
Затем идёт N строк, в которых описываются узлы связного списка в порядке увеличения индекса. Описание каждого узла состоит из его строкового значения и индекса следующего узла в списке (или -1, если следующего нет).

Наконец, в тесте идут Q строк, которые описывают операции над списком. В каждой строке сначала записан тип операции: 0 — добавление, 1 — удаление. Затем указан индекс узла, после которого нужно вставить/удалить узел. Если описывается операция вставки узла, то в конце также задано строковое значение нового узла.
У каждого узла строковое значение имеет длину от 1 до 7 символов включительно, и состоит из произвольных печатаемых символов ASCII кроме пробела (такие символы имеют код от 33 до 126 включительно).
Сумма N по всем тестах не превышает 105, аналогично для суммы всех Q.

Формат выходного файла

Для каждого теста нужно вывести следующее:

1. результаты выполнения операций (Q строк)
2. строка "===" (три знака равенства)
3. строковые значения всех узлов списка после выполнения операций (в порядке их следования в списке)
4. снова строка "==="

Примеры

input.txt	output.txt
3 5 3 0 4.283 2 2.718 4 5.0 -1 1.111 1 3.141 0 1 0 5 zero -1 0 -1 one 0 -1 two 0 1 three 0 0 four 1 2 4 2 2 45 1 %drill# 3 &qw6: 0 *a-+r -1 1 1 0 2 \num\	=== 1.111 2.718 3.141 4.283 5.0 === 1 2 3 4 one === two three zero four === *a-+r 4 === &qw6: \num\ 45 %drill# ===

Пояснение к примеру

Первый тест полностью совпадает с примером к задаче “Распечатать список”: дано 5 узлов и 0 операций. Т.к. операций нет, в ответе записана сразу строка "===". Потом записан ответ как в задаче “Распечатать список” и ещё одна строка "===".

Во втором тесте изначально есть только один узел. Заметим, что в каждом узле значение равно индексу, записанному по-английски: zero, one, two, three, four.

Первые четыре операции задают вставку элементов: сначала вставляется два узла в начало, получается список two, one, zero. Затем вставляется узел после узла one и ещё один узел после узла zero, получается список two, one, three, zero, four. Последняя операция удаления: удаляется узел, который стоит после узла two, то есть узел one.

Задача 3. Сортировка со списками

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В первой строке записано целое число N — количество записей ($1 \le N \le 2 \cdot 10^5$). В остальных N строках содержатся записи, по одной в строке.
Для каждой записи указаны ключ и значение через пробел. Ключ — это целое число в диапазоне от 0 до 106 включительно, а значение — это строка от одного до семи символов включительно, состоящая только из маленьких букв латинского алфавита.
Требуется вывести ровно те же самые N записей, но в другом порядке. Записи должны быть упорядочены по возрастанию ключа. Если у нескольких записей ключ равный, то нужно упорядочить их в том порядке, в котором они встречаются по входном файле.
Важно: Решать задачу **нужно** следующим образом (другие решения засчитываться **не** будут). Нужно завести 10^6 связных списков, и в каждый k-ый список складывать все записи с ключом, равным k. Тогда после раскидывания записей по спискам достаточно будет пробежаться по спискам в порядке увеличения k и распечатать их.

Пример

input.txt	output.txt
7 3 qwerty 3 string 6 good 1 a 3 ab 2 hello 5 world	1 a 2 hello 3 qwerty 3 string 3 ab 5 world 6 good

Пояснение к примеру

В примере 7 записей с ключами 1, 2, 3, 5 и 6 — именно в таком порядке записи и выведены в выходном файле. Обратите внимание, что есть три записи с ключом 3: qwerty, string, ab. Они выведены ровно в том порядке, в котором они идут во входном файле.

Задача 4. Двусвязный список

Источник:	базовая
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Требуется реализовать двусвязный список. В каждом узле списка хранится целочисленное значение и индексы следующего и предыдущего узлов. Дано начальное состояние списка, а также последовательность операций двух видов: добавление и удаление узла. Нужно выполнить все операции, и после этого вывести значения всех узлов списка в порядке их следования в цепи.

У каждого узла списка есть индекс в едином массиве, в котором всё хранится. В начальном состоянии списка есть N узлов, имеющих индексы от 0 до $N - 1$ в порядке их задания. При добавлении нового узла он дописывается в конец массива. То есть первый добавленный узел имеет индекс N, второй добавленный — $N + 1$, и так далее.

Нужно выполнять операции двух видов:

1. *Добавление узла спереди.* При этом указывается индекс узла, после которого нужно вставить новый узел, и значение для нового узла. Если индекс равен -1, то узел надо вставить в начало списка (перед самым первым элементом). После выполнения операции нужно вывести индекс нового узла (они назначаются по порядку, см. выше).

-1. *Добавление узла сзади.* При этом указывается индекс узла, перед которым нужно вставить новый узел, и значение для нового узла. Если индекс равен -1, то узел надо вставить в конец списка (после самого последнего элемента). После выполнения операции нужно вывести индекс нового узла.

0. *Удаление узла.* При этом указывается индекс узла, который нужно удалить. После выполнения операции нужно вывести значение удалённого узла.

В задаче используется “мультитест”: в одном входном файле записано много отдельных тестов.

Подсказка: Возможно, удобнее будет хранить связный список в виде двунаправленного кольцевого списка со вспомогательным элементом, например, сдвинув нумерацию узлов в программе на один.

Формат входного файла

В первой строке файла записано одно целое число T — количество тестов в файле. Далее в файле идут тесты (T штук) подряд, один за другим.

Первая строка теста начинается с целых чисел: N — изначальное количество узлов в связном списке, F — индекс первого элемента списка, L — индекс последнего элемента списка и Q — количество операций, которые нужно выполнить ($0 \le F, L < N \le 10^5$, $0 \le Q \le 10^5$).

Затем идёт N строк, в которых описываются узлы связного списка в порядке увеличения индекса. Описание каждого узла состоит из его целочисленного значения и двух индексов: следующего узла в списке и предыдущего (или -1, если одного из них нет).
Наконец, в тесте идут Q строк, которые описывают операции над списком. В каждой строке сначала записан тип операции: 1 — добавление спереди, -1 — добавление сзади, 0 — удаление. Затем указан индекс узла. Если описывается операция вставки, то в конце также задано целочисленное значение нового узла.
Все значения узлов лежат в диапазоне от 0 до 10^6 включительно.
Сумма N по всем тестах не превышает 10^5, аналогично для суммы всех Q.

Формат выходного файла

Для каждого теста нужно вывести следующее:

1. результаты выполнения операций (Q строк)
2. строка "===" (три знака равенства)
3. целочисленные значения всех узлов списка после выполнения операций (в порядке их следования в списке)
4. снова строка "===".

Пример

input.txt	output.txt
3 5 3 2 0 4283 2 4 2718 4 3 5000 -1 0 1111 1 -1 3141 0 1 1 0 0 5 0 -1 -1 1 -1 1000 -1 -1 2000 1 1 3000 -1 0 4000 0 2 4 2 3 2 45 1 2 74 3 0 16 0 -1 56 -1 1 0 1 1 2 35	=== 1111 2718 3141 4283 5000 === 1 2 3 4 2000 === 1000 3000 4000 0 === 74 4 === 16 35 45 56 ===

Пояснение к примеру

В примере три теста, похожие на тесты из задачи “Односвязный список”. Во всех тестах заменены строковые значения на целочисленные, во втором тесте также другой порядок узлов.

Задача 5. Разделить на слова

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

Требуется реализовать функции для выделения слов из заданной строки. Слова отделены друг от друга символами-разделителями, которые передаются в строку как параметр. Сигнатура функций должна быть такой:

```
void tokensSplit(Tokens *tokens, const char *str, const char *delims);
void tokensFree(Tokens *tokens);
typedef struct Tokens_s
{
    int num;    // количество слов в строке
    char **arr; // массив слов (каждый элемент -- строка, т.е. char*)
} Tokens;
// tokens: структура, в которую нужно положить результаты
// str: строка, в которой нужно искать слова
// delims: все символы-разделители в виде строки
void tokensSplit(Tokens *tokens, const char *str, const char *delims);
// удаляет все ресурсы внутри tokens
void tokensFree(Tokens *tokens);
```

Память для массива слов tokens->arr следует выделять динамически ровно на столько элементов, сколько слов в строке. Поскольку заранее это количество неизвестно, то нужно запускать алгоритм два раза: первый раз, чтобы узнать сколько слов, и второй раз, чтобы записать слова в массив.

Ваша реализация вышеописанных функцию должна работать согласно следующим договорённостям:

1. Вызывающий гарантирует, что параметр tokens не нулевой (для обеих функций) и указывает на структуру Tokens.
2. Если при вызове tokensSplit указатель tokens->arr нулевой, то внутри функции нужно только посчитать количестов слов и записать его в tokens->num.
3. Если при вызове tokensSplit указатель tokens->arr не нулевой, то он должен указывать на массив, в который точно войдут все слова. В этом случае реализация функции должна записать в tokens->num количество слов, а в tokens->arr[i] записать i-ое слово, самостоятельно выделив под него память с помощью malloc.
4. Функция tokensFree должна удалять массив слов и сами строки-слова с помощью free. При этом программа должна работать корректно, даже если эту функцию случайно вызовут два или три раза подряд.

Таким образом, вызывающий может завести структуру tokens, потом определить количество слов первым вызовом tokensSplit, затем выделить память на массив tokens->arr, и, наконец, найти все слова вторым вызовом tokensSplit.

С помощью этих функций нужно решить тестовую задачу. Дана одна строка длиной до 10^6, состоящая из букв латинского алфавита и знаков препинания четырёх типов: точка, запятая, точка с запятой, двоеточие. Нужно найти слова в этой строке, состоящие из букв, и вывести их в файл в таком же формате, как показано в примере.

Пример

input.txt	output.txt
..ko,.Privet:kreved,.,;ko:;,.	4 ko Privet kreved ko

Комментарий

Следует выводить слова ровно в том порядке, в котором они встречаются в строке.

Задача 6. Список по указателям

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В данной задаче нужно реализовать набор функций для работы со связным списком, так чтобы пользователь оперировал напрямую указателями на лежащие внутри узлов значения. Список должен быть двусвязный, кольцевой со вспомогательным узлом. Каждый узел списка должен быть размещён в динамической памяти, выделенной с помощью malloc.
Нужно реализовать следующие функции:

```
typedef struct Node
{
    struct Node *prev, *next;
    double value;
} Node;
// создаёт пустой список
// возвращает указатель на value в голове списка (вспом. узел)
double *initList();
// полностью освобождает память списка
// принимает указатель на value голове списка (вспом. узел)
void freeList(double *head);
// определяет следующий элемент после заданного
//(для вспом. узла возвращает первый элемент)
double *getNext(double *curr);
// определяет предыдущий элемент перед заданным
//(для вспом. узла возвращает последний элемент)
double *getPrev(double *curr);
// создаёт и вставляет узел со значением newval сразу после узла where
double *addAfter(double *where, double newval);
// создаёт и вставляет узел со значением newval сразу перед узлом where
double *addBefore(double *where, double newval);
// удаляет и освобождает заданный узел
void erase(double *what);
```

Для реализации этих функций нужно уметь находить указатель на узел по указателю на значение value внутри него.
Используя эти функции, решите тестовую задачу, аналогичную задаче “Список с указателями” (см. набор задач по теме связных списков).

Формат входного файла

В первой строке файла записано одно целое число T — количество тестов в файле. Далее в файле идут тесты (T штук) подряд, один за другим.

Первая строка теста начинается с целого числа Q — количество операций, которые нужно выполнить ($0 \le Q \le 10^5$).
Затем идут Q строк, которые описывают операции над списком. В каждой строке сначала записан тип операции: 1 — добавление спереди, -1 — добавление сзади, 0 — удаление.
Затем указан индекс узла. Если описывается операция вставки, то в конце также задано вещественное значение нового узла.
Все значения узлов заданы максимум с тремя знаками после десятичной точки, и по модулю не превышают 10^4.
Сумма Q по всем тестах не превышает 10^5.

Формат выходного файла

1. При выполнении операции удаления узла выводите значение удаляемого узла
2. После выполнения всех операций выведите строку ===
3. Затем выведите значения всех оставшихся узлов списка в порядке их следования
4. Наконец, выведите ещё раз строку ===

Все вещественные числа следует выводить с помощью формата "%0.3lf".

Пример

input.txt	output.txt
2 5 1 -1 4283 -1 0 2718 1 0 5000 -1 1 1111 1 1 3141 9 1 -1 0 1 -1 1000 -1 -1 2000 1 1 3000 -1 0 4000 0 2 1 0 9876.543 0 1 0 3	1111.000 2718.000 3141.000 4283.000 5000.000 === 2000.000 1000.000 3000.000 === 4000.000 0.000 9876.543 ===

Задача 7. Список с индексами

Источник:	основная
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

У связного списка есть серьёзная проблема: в отличие от массива в нём нельзя быстро получить k-ый по порядку элемент. Это можно сделать лишь перебором узлов списка за $O(k) \approx O(N)$ времени. В данном задаче предлагается ускорить поиск узла по индексу. Рассмотрим односвязный список. Пусть в списке помимо обычных узлов (“маленьких” узлов) есть ещё немного особенных узлов (“больших” узлов). В любом узле списка нужно хранить значение узла и указатель на следующий по порядку элемент. В каждом большом узле списка предлагается дополнительно хранить указатель на следующий большой узел списка и расстояние до него. Под расстоянием здесь понимается то, сколько переходов вперёд нужно сделать из одного узла, чтобы попасть по второй узел.

Дополнительные ссылки в больших узлах позволяют быстрее найти k-ый элемент, так как можно проскакивать сразу по много узлов, изначально проходя исключительно по большим узлам. Можно заметить, что если доля больших узлов в списке примерно $\frac{1}{B}$, то найти k-ый элемент можно примерно за $O(\frac{k}{B} + B)$. (Вопрос: как лучше выбрать B?)

При добавлении элементов важно поддерживать указанную выше структуру. Вставку узла будем выполнять по индексу, на котором должен стоять новый элемент. При этом сначала нужно решить, будет ли новый узел большим — это можно делать случайным образом, так чтобы поддерживать желаемую долю больших узлов. Затем нужно найти по индексу последний больший и малый узлы перед тем местом, куда нужно вставить новый узел. Наконец, можно вставить новый узел, корректно обновив все ссылки и расстояния так, чтобы сохранилась структура списка.

Предлагается реализовать эту структуру данных, и обработать с её помощью серию из N запросов двух типов (в описаниях L — текущее количество узлов в списке):

0. Вставить на k-ую позицию новый узел с заданным значением V. Гарантируется, что во всех запросах k лежит в пределах от 0 до L.

1. Вывести значение узла, находящегося на k-ой позиции в списке. Гарантируется, что во всех запросах k лежит в пределах от 0 до $L - 1$.

Решение в целом должно работать за время $O(N\sqrt{N})$.

Формат входного файла

В первой строке задано целое число N — общее количество запросов ($1 \le N \le 10^5$).
В каждой из следующих N строк дан один запрос. Запрос вставки задаётся тремя целыми числами $0\ k\ V$, а запрос на вывод элемента задаётся двумя целыми числами $1\ k$. Все числа V в списке целые, по модулю не превышают 10^9.

Формат выходного файла

Для каждого запроса на вывод (типа 1) нужно вывести значение k-ого узла в списке.

Пример

input.txt	output.txt
10 0 0 7 0 0 5 0 1 3 0 0 2 1 0 1 1 1 2 1 3 0 1 -5 1 2	2 5 3 7 5

Комментарий

Несмотря на формально лучшую асимптотику, полученная структура данных будет обрабатывать заданные запросы не быстрее простого массива.

Задача 8. XOR-список

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	разумное

В узле односвязного списка хранится один указатель, но по такому списку нельзя перемещаться в обратную сторону и неудобно удалять элементы. В двусвязном списке всё просто и красиво, но приходится хранить уже по два указателя в каждом узле. Данная задача посвящена очень странной конструкции, когда в каждом узле хранится только одно дополнительное значение, но при этом можно перемещаться в обе стороны.

Как известно, указатель — это просто адрес в памяти, а над адресами можно выполнять арифметические операции. В узле предлагаемого списка вместо указателей next и prev нужно хранить, к примеру, сумму этих двух адресов. Тогда если в программе известны указатели на два подряд идущих элемента такого списка left и right, то адрес соседнего узла можно найти вычитанием адреса одного узла из хранящейся в другом узле суммы адресов соседей. Общий принцип заключается в том, что для выполнения любой операции (даже перемещения) в таком списке необходимо иметь два указателя, которые смотрят на два подряд идущих элемента списка, потому что по одному указателю невозможно найти соседние узлы.

Канонически, вместо суммы адресов соседей в узле обычно хранят “побитовое исключающее или” (т.е. XOR) адресов соседей. Тому есть несколько причин: во-первых, операция XOR играет роль сложения и вычитания одновременно; во-вторых, операция XOR порой выполняется немного быстрее сложения и вычитания, т.к. её проще реализовать на аппаратном уровне. Канонический вариант XOR-списка описан в википедии.

Требуется реализовать XOR-список и решить тестовую задачу. Примерно, как в задаче “Список по указателям”, но теперь в узлах надо хранить целочисленные значения.

Нужно реализовать следующие функции:

```
#define VALTYPE int // тип значений: целое
typedef struct Node_s
{
    size_t xorlinks; // XOR адресов соседних узлов
    VALTYPE value;   // значение в этом узле
} Node;
typedef struct List_s
{
    ??? // тут можно хранить всякие данные
} List;
// инициализировать пустой список в list
void initList(List *list);
// добавить новый узел со значением val между соседними узлами left и right
Node *addBetween(Node *left, Node *right, VALTYPE val);
```

```
// дано два соседних узла left и right, нужно удалить узел left
VALTYPE eraseLeft(Node *left, Node *right);
// дано два соседних узла left и right, нужно удалить узел right
VALTYPE eraseRight(Node *left, Node *right);
```

Для простоты реализации рекомендуется сделать XOR-список кольцевым с двумя вспомогательными элементами (“начало” и “конец”)..

Формат входного файла

В первой строке файла записано одно целое число T — количество тестов в файле. Далее в файле идут тесты (T штук) подряд, один за другим.

Первая строка теста начинается с целого числа Q — количество операций, которые нужно выполнить ($0 \le Q \le 10^5$). В начале каждого теста список пустой.

Затем идут Q строк, которые описывают операции над списком. В каждой строке сначала записан тип операции: 1 — удаление правого, -1 — удаление левого, 0 — добавление. Затем указано два индекса: для узла left и узла right. Если описывается операция вставки, то в конце также задано целочисленное значение нового узла.

Гарантируется, что указанный узел right всегда идёт сразу после указанного узла left. Один из этих узлов, может быть, не указан (тогда записан индекс -1), если выполняется операция на краю списка.

Все значения узлов лежат в диапазоне от 0 до 10^6 включительно.

Сумма Q по всем тестам не превышает 10^5.

Формат выходного файла

Для каждого теста нужно вывести строковые значения всех узлов списка после выполнения операций (в порядке их следования в списке), и строку "===" в конце.

Пример

input.txt	output.txt
2 5 0 -1 -1 4283 0 -1 0 2718 0 0 -1 5000 0 -1 1 1111 0 1 0 3141 7 0 -1 -1 0 0 -1 0 1000 0 0 -1 2000 0 1 0 3000 0 2 -1 4000 1 0 2 -1 0 4	1111 2718 3141 4283 5000 === 1000 3000 4000 ===

Задача 9. Список с индексами++

Источник:	повышенной сложности
Имя входного файла:	input.txt
Имя выходного файла:	output.txt
Ограничение по времени:	1 секунда
Ограничение по памяти:	8 мегабайт

Данная задача посвящена дальнейшему улучшению структуры данных из задачи “Список с индексами”.
Вместо деления узлов лишь на “большие” и “маленькие”, припишем каждому узлу “высоту”. Высота определяет, сколько в узле хранится ссылок на следующие элементы. В предыдущей задаче список имел только два уровня: “маленькие” узлы имели высоту 1, а “большие” — высоту 2. В этой задаче предлагается завести 20 уровней (на самом деле примерно $\log_2 N$), распределяя высоты от 1 до 20.
В узле высоты h нужно хранить массив из h указателей вперёд вместе с расстояниями.
k-ый указатель смотрит на следующий в списке элемент, высота которого хотя бы k (здесь полагаем, что k считается с единицы). Для идеальных результатов узлов высоты k должно быть примерно $2{-}k$ по доле. Этого можно достичь, если при добавлении каждого нового узла бросать монетку до тех пор, пока не выпадёт орёл, а высоту устанавливать, как количество выполненных бросков.
При правильной реализации этой структуры можно выполнять поиск по индексу и добавление узлов примерно за $O(\log N)$. Такая структура называется “список с пропусками” / “skip lists”: ссылка.
В данной задаче нужно обработать N таких же запросов, как в задаче “Список с индексами”, но теперь за время $O(N \log N)$.

Формат входного файла

В первой строке задано целое число N — общее количество запросов ($1 \le N \le 10^6$).
В каждой из следующих N строк дан один запрос. Запрос вставки задаётся тремя целыми числами $0\ k\ V$, а запрос на вывод элемента задаётся двумя целыми числами $1\ k$. Все числа V в списке целые, по модулю не превышают 10^9.

Формат выходного файла

Для каждого запроса на вывод (типа 1) нужно вывести значение k-ого узла в списке.

Пример

input.txt	output.txt
10 0 0 7 0 0 5 0 1 3 0 0 2 1 0 1 1 1 2 1 3 0 1 -5 1 2	2 5 3 7 5